Q13: How Models Represent and Store Knowledge

Paradise (nuc13)

October 2026

The question

- "How do models represent and store knowledge, and use it to answer?"
- No database, no schema — yet facts come out. Where do they live?
- Answer preview: **mid-layer FFN modules as key–value memories; features as directions in superposition; retrieval routed by attention**

Storage site: FFN layers are key–value memories

- FFN $= \approx$ two-thirds of Transformer parameters
- $\mathrm{FFN}(x) = f(xK^\top)\,V \quad \Rightarrow$ structurally a key–value memory (Geva et al. 2021)
- Keys: rows of K match textual patterns; values: columns of V induce output distributions
- Lower layers: shallow patterns; upper layers: semantic themes
- Output $=$ weighted *composition* of memories, refined via residual stream

Retrieval path: causal tracing (ROME)

- Intervention: corrupt the input; restore states one at a time; see which repairs the fact
- Finding (GPT-2 XL): mid-layer **MLP modules** at the **last subject token** mediate recall
- Late-layer **attention** then moves the information to the output position
- Storage (MLP) and routing (attention) are *separable* mechanisms

Format: linear representations

- Linear representation hypothesis: features = **directions**; activations = $\sum_i a_i f_i$
- Why probing works: linear readout suffices — if information is used or at least accessible
- Caveat: probing accuracy $\neq$ causal use; complement with interventions
- But then: a d-dimensional layer could only hold d features...

Format: superposition

- Networks store **more features than dimensions** (Elhage et al. 2022)
- Trick: non-orthogonal, nearly-collapsed directions; high dimensions make random vectors almost orthogonal
- Cost: **interference** — co-active features leak into each other's readout
- ReLU acts as the noise filter; geometric structures (triangles, simplices) emerge
- **Polysemanticity** is the visible symptom — one neuron, many concepts

Demo: superposition in NumPy

- N features into $d < N$ dimensions via random near-orthogonal vectors
- Sparse features: recoverable readout despite interference
- Dense features: interference dominates — capacity exceeded
- `code/superposition_demo.py` — pass/fail assertions, no dependencies beyond NumPy

Editing: locate, then patch

- **ROME**: rank-one update of one mid-MLP matrix; $k_* \mapsto v_*$
- Generalizes across paraphrases, stays specific to the subject
- **MEMIT**: spreads batches of edits across mediating layers (thousands at once)
- Side effects: bleedover to neighbors, fluency degradation, brittleness
- Open problem: surgical edits in a superposed store, with guarantees

In weights vs. in context

- Weights: vast, static, hard to update (edit/retrain)
- Context: tiny, instant, attention is the read mechanism
- Failure modes differ: stale/absent vs. retrieval error / overflow
- Forward-link to Q12: what happens when the two channels **conflict**

Reading

- Geva et al. 2021 — Transformer Feed-Forward Layers Are Key-Value Memories (EMNLP)
- Meng et al. 2022 — Locating and Editing Factual Associations in GPT (NeurIPS)
- Elhage et al. 2022 — Toy Models of Superposition (Transformer Circuits)
- Meng et al. 2023 — Mass-Editing Memory in a Transformer (ICLR)